

PART FIVE

Achieving Performance and Reliability with Relational Database Systems

This part looks at a collection of important problems that we face in the production of efficient and reliable database applications. The primary topics covered here are strategies for executing relational queries, collecting database operations into transactions, allowing concurrent transactions, providing backup and recovery, and enforcing security. The support for these operations by database systems makes them invaluable to information systems. You will learn the basics of how to enhance your applications to improve their performance and reliability. Special emphasis is placed on the ways in which Oracle database products deal with the issues of the section.

In Chapter 13, you will learn how a relational database server processes queries. The discussion first considers how the processing of simple queries takes advantage of the physical representation of database tables as file structures, as was described in Part Four. You will get a taste of what the implementers of relational database systems must consider in developing query-processing software. The chapter concludes with a discussion of how information about the contents of tables is used to create optimal query execution strategies.

Chapter 14 delves into the fascinating subject of transaction processing. A transaction is a container that surrounds a collection of database operations. The group of operations is treated as a single, atomic operation by the database server. You will learn how transactions allow an application to cancel a collection of updates through a rollback operation and how database servers typically support rollback. The most interesting aspect

of this topic is in the reliable and consistent processing of multiple concurrent transactions. Here we learn how to structure operations to ensure correctness and how systems ensure correctness of multiple transactions.

Chapter 15 addresses issues of reliability, backup, recovery, and security. You will learn how database servers create truly reliable databases through the use of backups and how systems can recover from failures of almost any kind. In addition, you will see how database systems support security by restricting access to database objects. The creation of users and roles and the granting and denying of privileges are described through an extensive array of examples. The reliability of distributed information systems is addressed in the context of distributed database systems and of managing transactions that are applied to multiple databases.

13

Query Processing and Query Optimization

CHAPTER OBJECTIVES

In this chapter, you will learn:

- Basic strategies for implementing relational operations
- Specific methods for processing simple selection and projection queries
- Several methods for processing join queries
- The nature and use of query plans and relational expressions
- Strategies for evaluating the execution speed of query plans
- The goals and procedures of query optimization
- Ways to analyze databases and optimize queries in Oracle8

In this chapter, we focus on how to process queries and how to optimize that processing. As we have seen in previous chapters, relational databases are stored in files. A row of a table is stored as a record in a file. A table may be stored in one or more files, or one or more tables may be stored in a single file. The primary organization is that discussed in Chapter 11. Each object (row) is turned into a sequence of bytes and then stored as a record in a file. Accessing a record requires finding its location in the file, reading the correct number of bytes, and unpacking those bytes into memory objects. Many of the issues in query processing, therefore, relate to file structures: how to represent the objects as records, how to support access to records by their content, and how to process the many sets of records that are required for complex join queries.

Query processing involves three steps. The first step is to find one or more plans for carrying out the processing. The second step is to estimate the costs and select the most

efficient plan. The third step is to carry out the execution. Without the first two steps, the third step can be guaranteed to be very slow.

Our presentation will begin with basic execution strategies for selection, projection, and join queries. Next, we will consider the equivalence of execution plans and see how we can use relational algebra to create execution plans for more complex queries. We'll see how information about the data content of a database can be used to improve the cost estimates for the plans and, therefore, to find the best plans. We conclude by considering how Oracle8 collects that information and uses it in query optimization.

In this chapter, we'll assume that each file contains one table, and each record in the file represents a row of the table. This assumption is a simplification, because all of the rows of all of the tables of a database may be stored as a single file. In most cases, however, not much difference can be found between having one file and having a set of files. In those cases where this distinction does matter, dividing the database into multiple files is generally a better approach.

Processing Selection Queries

13.1

Now that we are able to read and write files and to access records by address, we are ready to consider processing select queries. Following the approach taken throughout this book, the discussion looks at general issues in terms of specific examples, working from simple to complex.

The processing of a query is illustrated in Fig. 13.1, where pentagons show the steps in the processing and gray arrows show some of the movement of information. The client sends a select statement (1) to the server to be processed. The server (2) reads the appropriate table (A in this case) and creates a temporary structure (shown

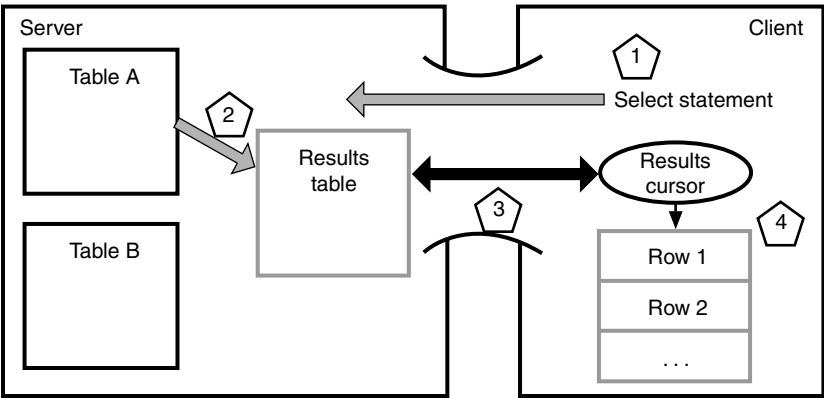


FIGURE 13.1 Query processing with a client and a server

with a gray border) to hold the select rows, either by value or by reference. The server sends a cursor (3) and some or all of the rows of the result back to the client. The black double-arrow between the results table and the cursor represents the interaction between those objects. More rows will be sent (4) as the client uses the cursor.

The results table may be a copy of all rows that satisfy the query, all addresses of the records that hold the result rows, or some kind of iterator that allows the rows to be found.

A *query plan* is a particular strategy for reading the indexes and files to process a query. The cost of a query plan is primarily determined by the number of blocks that must be read from the database files. The examples given in this section use the **Customer**, **Rental**, and **Movie** tables. Table 13.1 shows the assumptions that we use for the physical characteristics of the tables. This information will be used in estimating the cost of particular query execution plans. In general, the cost estimation is the number of block reads of indexes and data files. The number of rows that match the selection condition is another factor in the estimate. Here we ignore the cost of writing the result set in these estimates, because every plan must write the same result set.

13.1.1 Selection Based on a Single Attribute

We begin looking at query processing and cost estimation for the simple selection queries of Table 13.2. Each query selects all the attributes of rows that match a

TABLE 13.1
Physical characteristics of sample database tables

Table	Entries	Number of Entries per Block	Number of Blocks	Index Fields	Index Type	Keys per Node	Depth of B+ Tree
Customer	10,000	10	1,000	accountId	B+ tree and ordered sequential file	100	3
				lastName	B+ tree	50	3
				zipcode	hash	100	
Rental	1,000,000	100	10,000	accountId	B+ tree	100	3
				movieId	B+ tree	100	3
				date	B+ tree	100	2
Movie	10,000	20	500	movieId	B+ tree and ordered sequential file	100	3
				title	B tree	20	4
				genre	Hash	100	

TABLE 13.2

Simple queries on the Customer table, their processing plans, and estimated costs

	Query	Query Plan	Cost for k Matches	Cost for 100 Matches
1	select * from Customer where accountId = 101	B+ tree search and read data block	4	Not applicable
2	select * from Customer where accountId >= 101 and accountId < 300	B+ tree search and data file scan	$3 + k/10$	13
3	select * from Customer where lastName < 'D'	B+ tree search and B+ tree leaf node scan plus random read	$2 + k/50 + k$	104
4	select * from Customer where lastName = 'Jones'	B+ tree search and B+ tree leaf node scan plus random read	$2 + k/50 + k$	104
5	select * from Customer where zipcode = 32306	Hash lookup plus random read	$k/100 + k$	101
6	select * from Customer where city < 'D'	Full data file scan	1,000	1,000
7	select * from Customer where zipcode >= 32300 and zipcode < 32400	Full hash table scan plus random read	$100 + k$	200

selection on a single attribute. The first query returns the single row whose primary key is 101. Queries 2, 3, 6, and 7 are *range queries* that return all of the rows with an attribute value in a specific range. Queries 4 and 5 select all rows with a specific value for an attribute; they return multiple rows because the attributes `lastName` and `zipcode` are not unique. The processing strategies and estimated costs are explained next.

Query 1 of Table 13.2 can be processed by searching the `accountId` index for the value 101, then reading the corresponding block from the data file and extracting the customer information. The cost of this operation is three block reads for the index (two if the root node is in memory), and one for the data file. Figure 13.2 shows a B+ tree and indexed sequential file organization for the `Customer` table. The gray arrows show the search path; the gray nodes are the ones that must be read. As in the examples in Chapter 12, the number of keys in the B+ tree records, the number of records in the data file blocks, and the total number of records in the data file have been made artificially small to make it easier to identify the patterns.

The processing of query 2, as illustrated in Fig. 13.2, begins by searching the `accountId` index for the value 101 and reading the corresponding data file block.

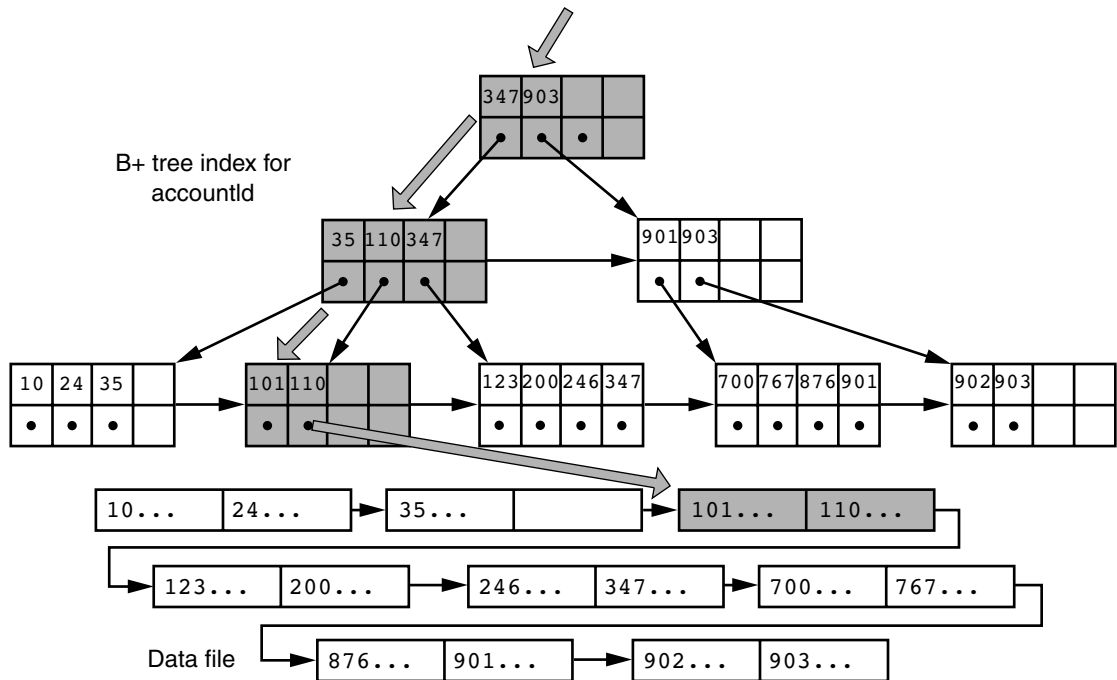


FIGURE 13.2
The query plan for query 1 of Table 13.2

It must read all of the customers whose `accountId` satisfies the range criterion. The `Customer` table is stored as a B+ tree and ordered sequential file on `accountId`, its primary key. Hence, the data file is logically stored in order by `accountId` as a linked list of blocks, as was described in the B+ tree and ordered sequential file example of Fig. 12.12. The processing continues by reading through the blocks of the linked list and extracting the customer records. As shown in Fig. 13.3, three data blocks must be read before an `accountId` larger than 300 is found. The processing stops after the third data file block is read. No further access to the index is required. The cost is three for the index blocks plus the number of blocks with records in the range. If the range has k keys, $k/10$ blocks must be read. The total cost is $3 + k/10$. If 100 records match the query, 13 blocks must be read.

The term *scan* is used in the table to refer to reading a sequence of records from a file. In the case of query 2, the data file is logically ordered by `accountId`. Once the first matching record is found (by B+ tree search), a scan is used to read the blocks in logical order until a record is found with an `accountId` that does not match. A *full scan* occurs when every block of a file is read.

Query 3 is a range query that requests all customers whose last name starts with a letter smaller than “D.” This query is significantly more expensive to execute than query 2 because the data file is not ordered by the selection attribute.

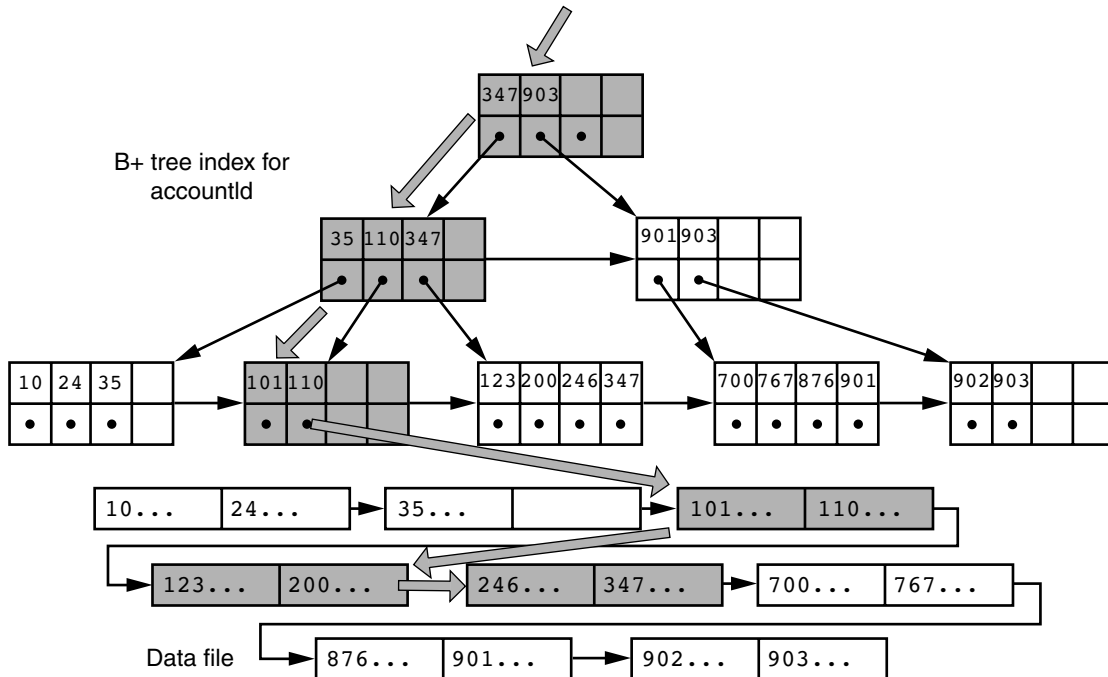


FIGURE 13.3
Select with B+ tree search and ordered data file scan

Consequently, the execution must search through the B+ tree index, finding all records that satisfy the range. Then each record must be read from the data file.

As illustrated by the gray arrows in Fig. 13.4, the search must read the root and one other interior node to reach the first node in the leaf level and find the smallest value for `lastName`. The index is ordered by `lastName`, so we can scan through the leaf nodes to find all records that satisfy the condition. For the small B+ tree of Fig. 13.4, we find a `lastName` greater than or equal to “D” in the second leaf node. The gray arrows leading from the leaf nodes indicate the reading of the data file records whose last name matches the select condition. Four records that match and the corresponding four data file blocks must be read.

For k matches, the B+ tree search and scan must read $k/50$ leaf nodes of the B+ tree. The selected records are distributed among the blocks of the data file in no particular order. Hence, it is likely that k blocks must be read. The total cost is estimated at $2 + k/50 + k$ reads. If 100 records match the query, 104 blocks must be read.

Figure 13.4 is, of course, an oversimplification. Here, all of the last names consist of two or three letters, and each last name has only one customer. The proper organization for a non-unique index on `lastName` is to have a block of

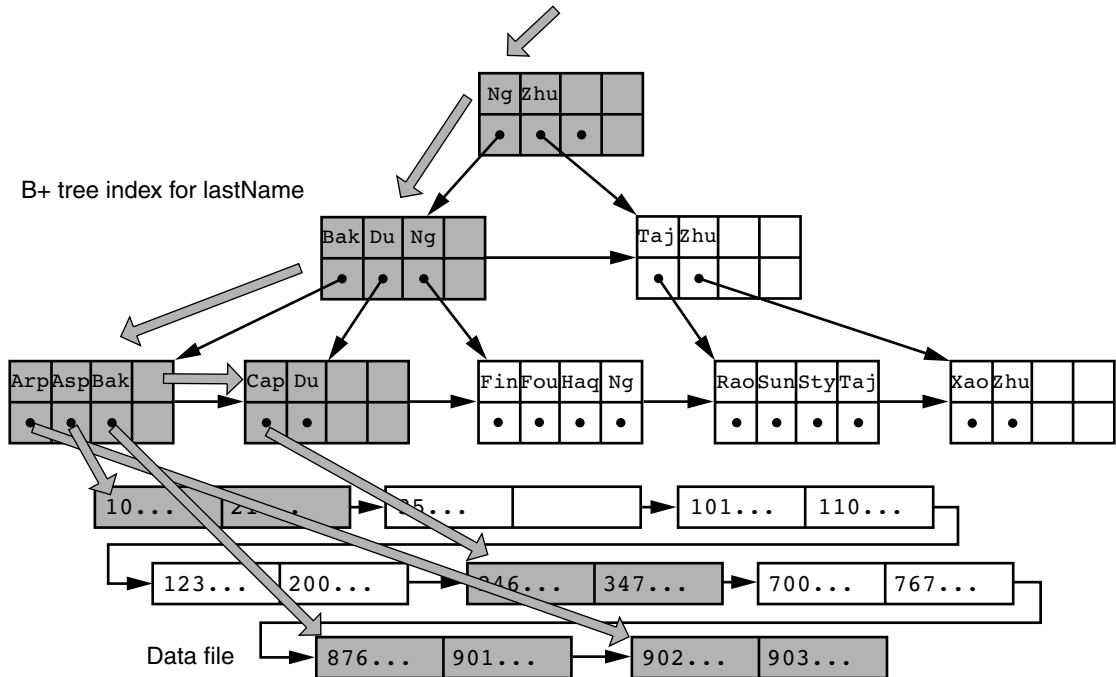


FIGURE 13.4
Select with B+ tree lookup and scan, and random data file reads

references for each key in the index. A reference block organization adds another read for each matching `lastName` value.

Although query 4 is not a range query, it returns a set of rows because the `lastName` attribute is not unique. Therefore, its expected execution cost is the same as that for query 3, even though the number of matching records (k) will likely be much smaller.

Query 5 is a selection based on a hash table index. Figure 13.5 illustrates the processing of query 5, with gray arrows to show the required reads. For k matches, we must read $k/100$ buckets to collect the addresses for all of the matching records. As with query 3, the data records are distributed among the data file blocks, so k data blocks must be read. The total cost is estimated at $k/100 + k$ reads. If 100 records match the query, 101 blocks must be read. The example of Fig. 13.5 shows a bucket with six references for the ZIP code 32306. After the bucket is found and read, each of the referenced data records must be read.

Query 6 is a range query on a non-indexed field. The only way to find matches to this query is to read every block in the data file. The cost is 1,000 read operations to read every block no matter how many records match. Even if only 100 records match the query, all 1,000 blocks of the data file must be read.

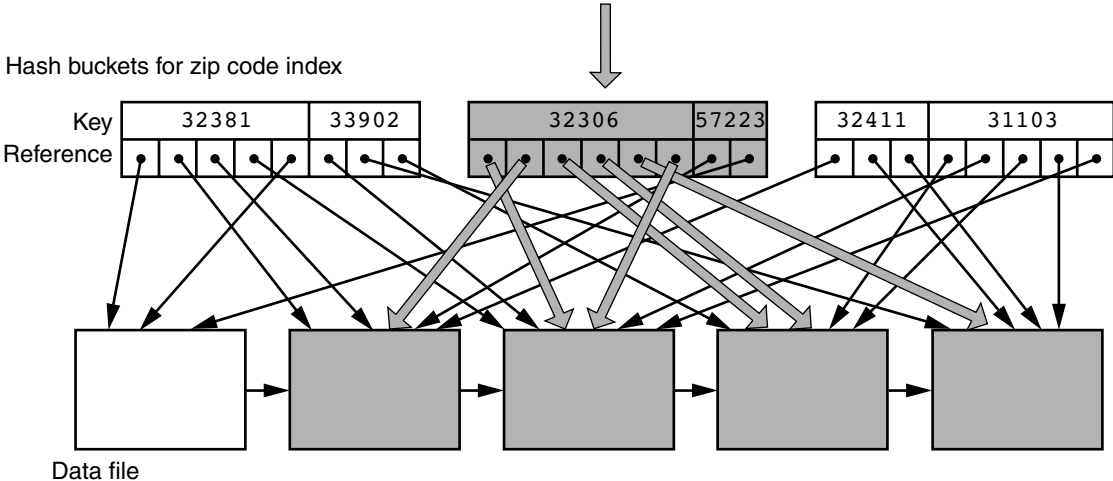


FIGURE 13.5
Select operation with hash table lookup and read

Query 7 is a range query on a field with a hash table index. The matches to this query can use the hash table, but only as a list of key values—not as a true index. Hash tables are not ordered by key, so the hashing function cannot be used to find all values in a range. The query processor must read every block in the hash table to identify the matching records. Because 100 blocks are in this hash file, the cost is 100 reads of the hash file and k reads of the data file. If 100 records match the query, 200 blocks must be read.

Figure 13.6 shows a simple example of using a hash table to satisfy a range query. The horizontal gray arrows show that all buckets are read. The gray arrows that point from the hash buckets to the data file blocks represent the data file reads. In this example, finding the records requires reading all of the data file blocks. In this very simplistic case, scanning through the hash table yields no advantage.

So far, we have identified several strategies for simple select query processing. Table 13.2 shows the best strategy for each query based on the availability of B+ trees, hash table indexes, and ordered data files.

13.1.2 Selection Based on Conjunctive Conditions

Processing (and cost estimation) is much more difficult with more complex selection conditions. In this section, we look at conditions involving more than one range of values or more than one attribute. The focus is on conjunctive queries—ones that involve a sequence of conditions connected by **and** (conjunction) operators.

Equivalence rules for relational algebra are crucial to finding query plans for these queries. Each query must be represented as a relational expression, and equivalent expressions must be considered. For instance, Table 13.3 lists a sample query, four equivalent relational algebra expressions, processing plans, and estimated costs.

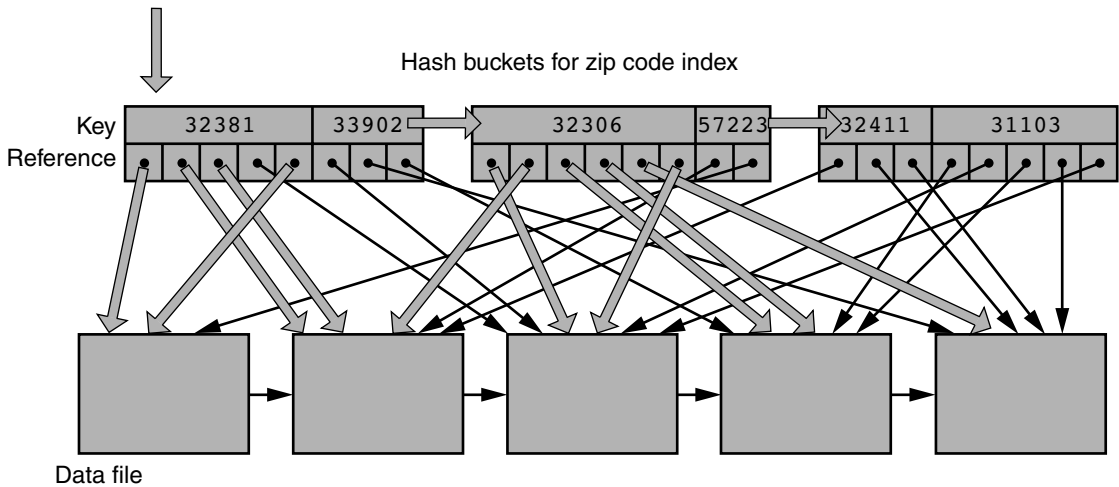


FIGURE 13.6
Select operation with hash table scan and read

TABLE 13.3
A query, equivalent expressions, query plans, and estimated costs

	<code>select * from Customer where accountId < 101 and lastName < 'D'</code>	Query Plan	Cost for k_1+k_2 Matches and k_3 Results
1	$\sigma_{\text{accountId}<101 \wedge \text{lastName}<'D'}(\text{Customers})$	Full data file scan, check each record for match	1,000
2	$\sigma_{\text{accountId}<101}(\text{Customers}) \cap \sigma_{\text{lastName}<'D'}(\text{Customers})$	B+ tree searches on accountId and lastName , collect set of record addresses, compute intersection, read data file blocks	$5 + k_1/10 + k_2/50 + k_3$ if few matches
3a	$\sigma_{\text{accountId}<101}(\sigma_{\text{lastName}<'D'}(\text{Customers}))$	B+ tree search on lastName , read each matching record, evaluate accountId , output if it matches	$2 + k_2/50 + k_2$
3b		If index on last name has accountId as reference, B+ tree search on lastName , evaluate accountId , search accountId B+ tree, read only fully matching records	$2 + k_2/50 + k_3$
4	$\sigma_{\text{lastName}<'D'}(\sigma_{\text{accountId}<101}(\text{Customers}))$	B+ tree search on accountId , read each matching record, evaluate lastName , output if it matches	$3 + k_1/10$

The costs are based on having k_1 matches to the `accountId` condition, k_2 matches to the `lastName` condition, and k_3 output rows.

Each of the expressions in Table 13.3 suggests one or more query plans. Expression 1 can be processed by directly selecting those rows that satisfy the entire condition. This approach requires a full data file scan. The query plan is to read a record and check the conditions. If both are satisfied, the record is added to the output set. This process continues until all records are read. This strategy will be cost-effective only if a very high percentage of the customers satisfy the conditions.

Expression 2 can be executed by performing the two selections separately and making sets of the record addresses for each selection, then computing the intersection of the two sets. The last step is to read the records that match both conditions. The individual selections are the same as in queries 2 and 3 of Table 13.2. Determining the record addresses takes $3 + k_1/10 + 2 + k_2/50$ block reads. Reading the matching data records takes k_3 block reads.

Computing the intersection costs nothing (in this cost model) if the number of matching records is small enough to fit in memory. The intersection is typically calculated by sorting the two sets separately and then comparing them in order.

If the number of matching records is too large to fit in memory, the plan calls for creating the set of record addresses for `accountId` matches as a searchable structure (for example, a B+ tree). As the `lastName` matches are found, the `accountId` matches are searched for the record address and marked as matching. Finally, all of the `accountId` matches that have been marked are read and written to the output file. The cost of this intersection is on the order of $k_1 + k_2$.

Expression 3 has two plans, 3a and 3b. Both are based on the selection of all customers by `lastName` and then selection among those by `accountId`. The cost of 3a is the case in which the index on `accountId` uses record addresses as references. The plan is to find all k_2 matches to `lastName`, read those k_2 records, and evaluate their `accountId` fields. The cost is the full cost of query 3 of Table 13.2: $2 + k_2/50 + k_3$ block reads.

The cost of 3b represents the case in which the index of `lastName` stores its references as primary keys. This strategy is explained in Section 12.2.1. The index selection returns the `accountId` values, which can be tested for the first condition. For each `accountId` in the range, the `accountId` B+ tree must be searched and the data record read. The cost of this selection will be the cost of searching the `lastName` ($2 + k_2/50$ blocks), searching for the `accountId` values (no more than k_3 leaf nodes), and reading only those data records whose `accountId` is in range (k_3 blocks). The total cost is therefore $2 + k_2/50 + 2k_3$ block reads.

The plan for expression 4 resembles plan 3a. All of the records that match the `accountId` condition are read, as in query 2 of Table 13.2, and then the `lastName` fields are evaluated. Only those whose last name is in the range are output. The cost is $3 + k_1/10$.

Evaluating the actual costs of the query plans to choose the best plan can be difficult. The cost of each plan depends on the contents of the database, which determines the relative number of matches of the two conditions. If k_1 is small relative to k_2 , expression 4 is the best because it takes advantage of the ordering of the data file (indexed sequential). If k_2 is small, however, expression 3 is the best.

The decision to use one of these plans must be made based on estimates of the values k_1 and k_2 . As we will see in Section 13.5, commercial DBMS systems provide facilities for estimating these values. Commercial query optimizers use this information to select plans for execution.

13.1.3 Selection Based on Disjunctive Conditions

Disjunctive queries involve a sequence of conditions connected by **or** (disjunction) operators. The basic strategies for evaluating these queries are the same as those given in the previous sections. The major additional problem is the elimination of duplicates.

The simplest disjunctive query is one based on multiple ranges of the primary key. An example of such a query is given in Table 13.4. The obvious processing strategy for expression 1 is to perform a full data file scan and test each row. This approach cannot be the best plan for this query, however.

TABLE 13.4

Disjunctive queries, equivalent expressions, plans, and estimated costs

	select * from customer where accountId < 101 or accountId > 1000	Query Plan	Cost for k_1+k_2 Matches and k_3 Results
1	$\sigma_{\text{accountId}<101 \vee \text{accountId}>1000}(\text{Customers})$	Full data file scan, test each record for full condition	1,000
2	$\sigma_{\text{accountId}<101}(\text{Customers}) \cup \sigma_{\text{accountId}>1000}(\text{Customers})$	B+ tree search and data file scan, twice	$5 + k_3/10$
	select * from customer where accountId < 101 or lastName < 'D'		
3	$\sigma_{\text{accountId}<101 \vee \text{lastName}<'D'}(\text{Customers})$	Full data file scan, test each record for full condition	1,000
4	$\sigma_{\text{accountId}<101}(\text{Customers}) \cup \sigma_{\text{lastName}<'D'}(\text{Customers})$	B+ tree search and scan, twice, union of record addresses plus read data file blocks	$6 + k_1/100 + k_2/50 + k_3$
	select * from customer where accountId < 101 or city < 'D'		
5	$\sigma_{\text{accountId}<101 \vee \text{city}<'D'}(\text{Customers})$	Full data file scan, test each record for full condition	1,000
6	$\sigma_{\text{accountId}<101}(\text{Customers}) \cup \sigma_{\text{city}<'D'}(\text{Customers})$	B+ tree search and data file scan plus full data file scan, test each record for city condition, union results	$3 + k_1/10 + 1,000$

Expression 2 describes the query as the union of two selections. The general strategy in this case is to perform the queries independently and eliminate the duplicates in the result sets. These conditions are exclusive, however, so no record satisfies both conditions. Expression 2 can be processed with two B+ tree searches and two data file scans. The two searches read a maximum of 5 B+ tree nodes, because the root node is read only once. The two data file scans read $k_1/10$ and $k_2/10$ blocks, respectively. The total number of matches is $k_3 = k_1 + k_2$, so approximately $k_3/10$ data file blocks must be scanned. The total number of blocks to read is $5 + k_3/10$.

The second query of Table 13.4 is a disjunction of two unrelated ranges. Again, the simple translation of the query to relational expression 3 results in an inferior plan. Expression 4 is the union of two selections. It can be processed in much the same way as expression 2. Assuming that the sets of record addresses for the two queries fit in memory, the cost of finding the matching record addresses is the cost of two B+ tree search and scans ($3 + k_1/100 + 3 + k_2/50$). After reading all of the matching records, the total cost is $6 + k_1/100 + k_2/50 + k_3$.

In a case where a field has no index, as in the third query of Table 13.4, the full data file must be scanned, as in the plan for expression 5. Using expression 6 yields no advantage.

Processing Projection Queries and Eliminating Duplicates

13.2

A projection operation eliminates columns and hence may create duplicates. No duplicates and no significant processing costs are incurred when the primary key, or any other key, is retained.

The techniques described in previous sections for eliminating duplicates in unions were based on eliminating duplicate record addresses or primary keys—that is, on ensuring that no row from the source table appeared more than once in the result set. These techniques will not work here, however. Instead, we must find duplicates by value, not by address. We will use the following simple projection query as our working example:

```
select distinct lastName, firstName from Customer
```

The two basic strategies for eliminating duplicates are based on sorting and on hashing. Both of the plans begin by creating an intermediate list of projected tuples. The first step is to scan the input data file and create a list by writing records with only the selected attributes. Then the plans diverge. The input to the next step for the above query is a list of records with `lastName` and `firstName` attributes. Suppose these records are smaller than the data records by a factor of t . The cost of the first step is the cost of a full scan of the data file (B blocks) plus the writing of the intermediate data file ($T = B/t$ blocks).

Each basic strategy can work by comparing all records or by dividing them into *partitions* and comparing within each partition. A partitioning strategy would be to divide the records by last name (or by part of the last name). Within a partition, each record has the same (or similar) last name. Any duplicates in the list will appear within the same partition. Partitioning has the potential to reduce the overall cost of the operation.

13.2.1 Eliminating Duplicates with Sorting

An effective query plan is to sort the list of projected records and write out only the nonduplicates. The basic cost is on the order of $T \log T$ (where T is the number of blocks in the temporary file) for the sorting and elimination of duplicates.

The sorting algorithm can be improved by having it eliminate duplicates during the sorting and by doing the projection as part of the first sorting step. The first improvement makes the sort operation yield the actual result set, thus eliminating a full read and write of the intermediate file. The second improvement eliminates the need to write and read the intermediate file.

If the first phase in the sort is to partition by a single attribute, the cost of sorting is reduced. The plan reads the intermediate file (T reads) and writes partitions (T writes). For p partitions, the sorting cost is on the order of $p \times (T/p \log T/p)$.

Projection by sorting is particularly useful when the query has an **order by** clause, as in Section 13.3, because the sorting phase can produce the results in output order with little loss of efficiency.

13.2.2 Eliminating Duplicates with Hashing

We can hash the entire intermediate file by applying a hash function to the concatenation of all attributes. This approach partitions the file into buckets of records that have the same hash value. As in the partition sort, the result is produced by sorting and eliminating duplicates within buckets. The cost is on the order of the number of hash buckets times the average size of the buckets plus the cost of sorting the buckets. It is quite cumbersome to calculate hash functions on very large string values such as will appear in this plan.

An improvement is to partition by hashing a single attribute. Under this plan, any records in different buckets will not be duplicates. Duplicates can be eliminated within a bucket by using an in-memory hash table with one or more different attributes. As each record is inserted into a secondary bucket, it is checked against all other records in the bucket. Duplicates are eliminated at this point.

The partitioned hashing plan must scan the intermediate file (T reads) and write the hash buckets (H writes, where $H \geq B$). It must then read the hash buckets (H reads) and write the result set. The cost for this phase— $T + 2H$ —is related to the number of hash buckets. With a perfect hash partitioning, the number of hash buckets is the same as the number of input blocks. It is likely, however, that some of the buckets will be much less than full and that the hash function will not produce a minimum number of buckets. This approach therefore results in a much larger number of buckets than input blocks.

Processing Join Queries

13.3

A join query combines two or more tables on matching attributes. With this type of query, the evaluation of query plans becomes more complex. In this section, the sizes of the tables will be represented symbolically instead of with specific numbers. Here we let R_c and B_c , R_r and B_r , and R_m and B_m be the number of rows and blocks of the **Customer**, **Rental**, and **Movie** tables, respectively.

13.3.1 Simple Nested Loops Join

Table 13.5 shows a sample join query and two query plans, represented as algorithms. The simplest and most obvious plans for the join operation are called *nested loops joins*. The code assumes that `customer` and `rental` are files that contain the input tables and that `result` is the file that will contain the output tables. This code has two versions; the only difference lies in which file is the inner loop and which is the outer one.

The number of reads in the inner loop of each plan is the product of the number of rows in the tables. Looking more carefully, we see that if the `Customer` table has R_c rows and B_c blocks and the `Rental` table has R_r rows and B_r blocks, the cost is $B_c + R_c \times B_r$ block reads for plan 1 and $B_r + R_r \times B_c$ for plan 2.

One plan is not obviously better than the other, because both appear to read the same number of data blocks. Suppose, however, that the entire `Rental` table fits into the disk cache of the system but the `Customer` table will not. Plan 1 will be much less expensive in this case.

With proper cache management, plan 1 will be able to keep the entire `Rental` table in memory during the execution of the algorithm. The first time through the

TABLE 13.5
Sample query and code for nested loops joins of `Customer` and `Rental`

select * from Customer c, Rental r where c.accountId = r.accountId	Number of Reads
1. Customer outer, Rental inner while (not customer.eof()) { c = customer.read(); rental.reset(); while (not rental.eof()) { r = rental.read(); if (c.accountId==r.accountId) { result.write(c, r); } } }	$B_c + R_c \times B_r$
2. Rental outer, Customer inner while (not rental.eof()) { row2 = rental.read(); customer.reset(); while (not customer.eof()) { row1 = customer.read(); if (c. accountId ==r.accountId) { result.write(c, r); } } }	$B_r + R_r \times B_c$

inner loop, every block of the `Rental` table will be loaded into disk cache. In subsequent executions of the inner loop, the read operations will access blocks already in memory with no actual disk access required. Thus the cost of plan 1 in this case is $B_c + B_r$, because each file must be read only once.

In plan 2, even though the `Rental` table always stays in memory, the `Customer` table must be read once for each execution of the inner loop. The cost of this plan is $B_r + R_c \times B_c$. Each block of the `Customer` table must be read once for each row of the `Rental` table.

Of course, if `Customer` fits in memory but `Rental` does not, plan 2 offers an advantage. In any case, the plan selection should depend on the relative sizes of the tables, the number of records per block, and the amount of memory available for disk caching. An improvement of these plans is described in the next section.

13.3.2 Block Nested Loops Join

The *block nested loops join* plan takes explicit advantage of record blocking. In the code shown in Fig. 13.7, method `readBlock` reads a block of records from the file and returns an array of objects. The inner loop then reads a record from the inner file and writes an output tuple for each object in the array. We don't have to explicitly deal with blocking in the inner loop, because the I/O system takes care of it for us. The cost of this algorithm is $B_c + B_c \times B_r$. Further improvements are available by reading multiple blocks in the outer loop, instead of just one.

13.3.3 Indexed Nested Loops Join

If one of the join attributes is indexed, the plan should be adjusted to take advantage of it. Table 13.6 shows two *indexed nested loops join* plans, using the `accountId` index of the `Rental` table for plan 1 and the index of the `Customer` table for plan 2. For each customer, plan 1 retrieves all of that customer's rentals, using an indexed read. Method `readByAcctId` returns an array of all `Rental` objects for a particular `accountId` value. Plan 2 uses the `Customer` index.

The two plans have different costs because of the one-to-many cardinality of the relationship type between the tables and because of the table contents. Each

```
while (not customer.eof()) {
    Customer c[] = customer.readBlock();
    rental.reset();
    while (not rental.eof()) {
        Rental r = rental.read();
        for (int i=0; i<c.length; i++) {
            if (c[i].accountId==r.accountId) {
                result.write(c[i],r);
            }
        }
    }
}
```

FIGURE 13.7
Query plan for block nested loops join for the query of Table 13.5

TABLE 13.6

Indexed nested loops query plans for the query of Table 13.5

	Number of Reads
1. rental inner <pre> while (not customer.eof()) { Customer c= customer.read(); rental.reset(); while (not rental.eof()) { Rental r[] = rental.readByAcctId(c.accountId); for (int i=0; i<r.length; i++) { result.write(c,r[i]); } } } </pre>	$B_c + R_r$
2. customer inner <pre> while (not rental.eof()) { Rental r= rental.read(); customer.reset(); while (not customer.eof()) { Customer c[] = customer.readByAcctId(r.accountId); for (int i=0; i<c.length; i++) { // length=1! result.write(c[i],r); } } } </pre>	$B_r + R_r$

customer has many rentals, but each rental has only one customer. Hence, plan 1 reads every customer once and every rental once. The rentals are not ordered by `accountId`, so a rental block may be read more than once. The cost is no more than $B_c + R_r$ block reads. One read occurs for each block of the `Customer` table, and no more than one read occurs for each row of the `Rental` table. Caching and the ordering of the records in the `Rental` table may greatly reduce the actual cost.

Plan 2 reads every `Rental` block and no more than one `Customer` block for each row of the `Rental` table. The number of reads cannot exceed $B_r + R_r$. The number of blocks in the `Customer` table is not a factor in this calculation, because each rental has a customer, but not every customer has a rental. Plan 2 will not read records for customers who have no current rentals.

Plan 1 scans the `Customer` file, then reads a block of the `Rental` file for every record in result set. Plan 2 scans the `Rental` file, then reads a block of the `Customer` file for every record in the result set. Because every rental has a customer, the number of records in the result set is exactly the number of records in the `Rental` table. In essence, plan 1 reads every customer and every rental once, but plan 2 reads some customers many times and some customers no times.

The decision about which plan to use can be made simply based on the size of the two files. Plan 1 is better if the `Customer` file is smaller; otherwise, plan 2 is better. Some insight into this decision-making process can be found by looking at the blocking factors of the two files. The `Rental` records are much smaller than the `Customer` records. If the `Rental` file is larger than the `Customer` file, many rentals must exist for each customer, and hence plan 1 is superior. Otherwise, the `Rental` file is smaller, each customer has relatively few rentals, and reading the customers multiple times is not too expensive.

If `Rental` were logically ordered by `accountId`—that is, represented as an ordered sequential file on `accountId`—the cost for both plans would be $B_c + B_r$. This solution amounts to reading each data file exactly once and ignoring the indexes, as explained in the next section.

13.3.4 Sort-Merge Join

A *sort-merge join* plan first sorts both files by their join attributes, then scans both sorted files together and merges the results. The cost will depend on the cost of sorting, which is typically on the order of $N \log N$ for a file of N records. In the case of the query of Table 13.5, however, the `Customer` file is already ordered by `accountId`. Only the `Rental` file must be sorted. If the `Rental` file is already ordered by `accountId`, the plan will consist of only the merge step. The cost will be that of scanning both files.

The basic merge algorithm (shown in Fig. 13.8), is initialized by reading a record from each file (lines 1 and 2). The main loop (lines 3–12) reads through the records in both files. A record is read from one of the files each time through the main loop. Whenever the current record from one of the files has no more matches, a new record is read from that file. When a customer–rental match is found (lines 4–7), the algorithm reads from the `Rental` file (line 6), but the customer record is retained and compared to the new rental on the next iteration. If the new rental has

```

1 Customer c = customer.read();
2 Rental r = rental.read();
3 while (true) {
4     if (c.accountId == r.accountId) {match
5         result.write(c, r); // done with r
6         r = rental.read();
7         if (r == null) break;
8     } else if (c.accountId < r.accountId) {// done with c
9         c = customer.read();
10        if (c == null) // error, no more customers, but more rentals
11    } else // error, no accountId matches the rental
12 } }
```

FIGURE 13.8
Algorithm to merge two sorted files

a larger `accountId` than the current customer (line 8), the customer has no more matches and a new customer is read (line 9). The algorithm terminates when end-of-file is reached in either file. Two error conditions occur (lines 10 and 11): when records are out of order, and when a rental does not match any customer.

The merge algorithm is an example of the general technique of *cosequential file processing*. The Further Readings section gives references to detailed treatments of this type of algorithm.

13.3.5 Hash Join

As described in Section 12.4.1, a hash function divides the values of some domain into a specific number of partitions. We can use this partitioning to greatly reduce the cost of some joins. A *hash join* creates a hash table from the join attributes of each input table, then compares the corresponding hash partitions for matches. All join matches will be located in corresponding buckets.

The processing of a hash join begins with the selection of a hash function that creates a specific number of partitions. The hash function should be selected so that a partition from each file can fit in memory.

1. Scan one input file and insert the join attribute–record address pairs into a hash table.
2. Create a second hash table from the other input file.
3. Read the first partition of each hash table into memory.
4. Compare the attribute values in the partitions to find all join matches.
5. For each join match, read the two corresponding records and output the merged record.
6. Repeat steps 3–5 for each additional hash partition.

The cost of the hash join depends on the sizes of the two hash tables (H_c and H_r). The cost of the hash join is the cost of reading the two files ($B_c + B_r$ block reads), writing the hash tables ($H_c + H_r$ block writes), reading the hash tables ($H_c + H_r$ block reads), and reading the data files again ($B_c + B_r$ block reads). The total is $2(B_c + B_r + H_c + H_r)$. The size of the hash tables is considerably smaller than the size of the data files, because only the join attribute and record address values are stored in the hash table.

13.3.6 Left, Right, and Outer Joins

The only additional processing for left joins, right joins, and outer joins is to include the records that don't match. Each of these algorithms is configured to output all matching tuples. It would not be difficult for sort-merge and hash joins to add all of the unmatched tuples from the left or right input tables to the result set.

The index nested loops joins require additional work for full outer joins. In particular, the algorithms must be changed so that the tuples are marked as they are matched, and a final scan of the table can be used to put all unmatched tuples in the result set.

A NOTE ON PROCESSING AGGREGATION QUERIES |

Aggregation operators in queries without **group by** clauses can be calculated by scanning the data file. Operators **min** and **max** can be calculated with the help of an index, if one is available.

An aggregation query with a **group by** clause requires the partitioning of records by the aggregation attributes. In such a case, hashing methods can be very effective for the same reasons that they are effective in hash join operations (Section 13.3.5) or projection (Section 13.2.2). A good hash function can create an initial partitioning scheme in which each group of records is clustered within a single partition. A partition may contain elements of multiple groups, but no group will span more than one partition. The grouping operations can be confined to individual partitions, thus greatly reducing the cost.

Query Plans and Query Optimization for Complex Relational Expressions

13.4

The previous sections looked at only the simplest queries. In those cases, it was usually obvious which query plans were possible and which were the best options. When multiple operations are combined into a single query, however, many query plans are possible and one may not be better than the others in all cases. This section provides a brief overview of the query optimization process, which consists of the following steps:

1. Enumerate the query plans.
2. Estimate the costs of the plans.
3. Choose the best plan for the query.

13.4.1 Creating Relational Expressions

A query plan is a relational expression that represents the query together with an evaluation strategy for each operator. The creation of a query plan begins with the translation of an SQL query into a relational algebra expression. This translation is not complicated (see the Further Readings listings at the end of the chapter for references to detailed algorithms).

Figure 13.9 shows three representations of a complex query that lists information about customer rentals. The query is represented as a select statement, a relational expression, and an expression tree.

The usual technique for producing query plans is to create equivalent expressions by applying the equivalence rules of the relational algebra. Associative, commutative, and distributive laws can be used to reorganize an expression. Figure 13.10 shows two expressions that are equivalent to the expression of Fig. 13.9. Figure 13.10a has moved the selection and projection operations lower in the tree. To do so, the selection operation was split into two selections: one by `lastName` and one by `dateDue` and `title`. The selection by `lastName` can be applied directly to the `Customer` table because it refers to an attribute that appears only in `Customer`. Similarly, the selections by `dateDue` and `title` can be applied to the

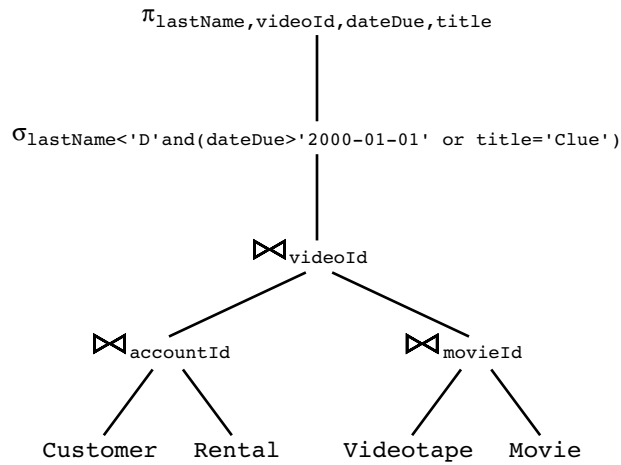


FIGURE 13.9
Example of an SQL query and its representation as an expression and a tree

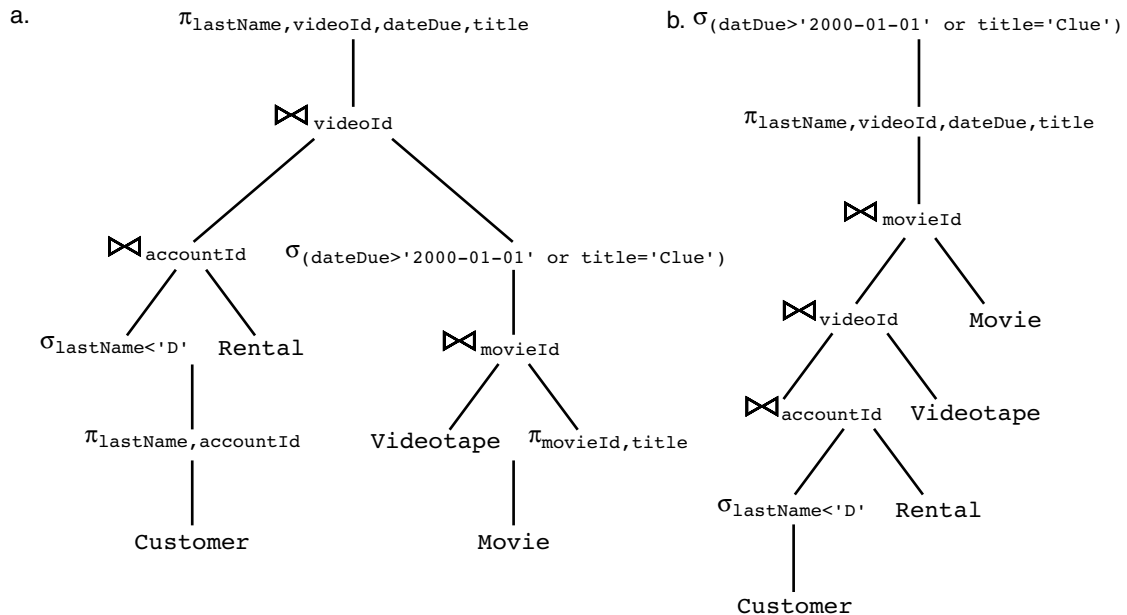


FIGURE 13.10
Expressions equivalent to those of Fig. 13.9

join of **Videotape** and **Movie** before this join is joined to the result of the join of **Customer** and **Rental**.

The projection operations cannot all be moved down the tree. In particular, **accountId** and **movieId** are required for the join operations and cannot be projected out of the data below the join operations. Instead, they must be eliminated by a projection near the top. The purpose of applying projection operations lower in the tree is to reduce the size of the records in the intermediate stages. Smaller records mean more records per block and hence fewer read and write operations.

Figure 13.10b shows an equivalent expression. The associative law of joins was used to modify the join operations so that **Customer** and **Rental** are joined, and this result is subsequently joined first to **Videotape** and finally to **Movie**. The selection and projection operations have been commuted at the top of the tree, so that now the projection is applied before the selection. Many other equivalent expressions exist as well.

Table 13.7 lists some of the basic rules of equivalence for relational expressions. The application of these rules produced the equivalent expression trees of Fig. 13.10. Similar rules exist for set operations and conditional expressions.

A query plan generator must use the equivalence rules to create expressions. A major concern of plan generators is to create all expressions that might produce efficient query plans and to not create those that won't produce such plans. Each query plan generator uses a collection of heuristics to decide which expressions to generate. As a consequence, not every expression will be generated. It is sufficient to know that it is very likely that no significant expression will be omitted. After all, if the *query optimizer* takes more time to execute than is saved by the optimal query plan, optimization has not made the execution more efficient.

TABLE 13.7

Some equivalence rules for relational algebra expressions

Equivalence Rule	Equivalent Expressions	
Cascading selections	$\sigma_{c_1 \wedge c_2 \wedge c_3 \wedge c_4}(R)$	$\sigma_{c_1}(\sigma_{c_2}(\sigma_{c_3}(\sigma_{c_4}(R))))$
Commuting selections	$\sigma_{c_1}(\sigma_{c_2}(R))$	$\sigma_{c_2}(\sigma_{c_1}(R))$
Cascading projections, for attribute sets $A_1 \subseteq A_2 \subseteq A_3$	$\pi_{A_1}(\pi_{A_2}(\pi_{A_3}(R)))$	$\pi_{A_1}(R)$
Commuting selection and projection, when condition c uses attributes in set A	$\sigma_c(\pi_A(R))$	$\pi_A(\sigma_c(R))$
Commuting joins	$R \bowtie S$	$S \bowtie R$
Distributing selection and join, when condition c_1 uses attributes in R and c_2 uses attributes of S	$\sigma_{c_1 \wedge c_2}(R \bowtie S)$	$(\sigma_{c_1}(R)) \bowtie (\sigma_{c_2}(S))$
Commuting projection and join, when attribute set A_1 has attributes from R and A_2 has attributes from S	$\pi_{A_1 \cup A_2}(R \bowtie S)$	$\pi_{A_1}(R) \bowtie (\pi_{A_2}(S))$
Associating joins	$R \bowtie (S \bowtie T)$	$(R \bowtie S) \bowtie T$

13.4.2 Pipelined Query Execution

The analysis of the cost of query execution for the examples of Sections 13.2 and 13.3 ignored the cost of writing the result set, because it was the same for every plan. The situation changes, however, when we consider the evaluation of multiple operations. A tremendous savings can be achieved if intermediate results are not written to files.

For instance, consider the join operations in the expression of Fig. 13.9. An indexed nested loops join can be used to join `Rental` and `Movie`. As each tuple is constructed from that join, the matching customer can be found by its `accountId` index. By *pipelining* these two joins, we avoid the expense of writing and reading the intermediate table. Similarly, the selection and projection operations can be applied to each tuple as the joins produce it. In this case, each input file is read as necessary to support the joins and the result is written with no intermediate files.

13.4.3 Constructing Query Plans from Expressions

A relational expression naturally determines an order of evaluation of the operations from bottom to top and from left to right, but does not specify how to evaluate each operation. A query plan is formed from an expression by choosing an evaluation strategy for each operation.

The processing strategy given in the example from the previous section represents a query plan that associates an indexed nested loops join with the lowest join operation, a pipelined indexed nested loops join for the join with `Customer`, and a pipelined scan, or *on-the-fly* evaluation, of the selection and projection operations.

The next step in query optimization, then, is to associate evaluation strategies with each node of each relational expression. This step usually results in more query plans than relational expressions. Exercises at the end of the chapter ask you to form query plans from the expressions of Figs. 13.9 and 13.10 and for additional queries.

13.4.4 Estimating the Cost of a Plan

Information used in cost estimation for plans includes the number of records, the size of the records in each table, the location and types of indexes, the ordering of files, the amount of memory available for disk cache and other memory structures, blocking factors, the number of levels of B+ trees, and more. The cost formulas given in Tables 13.2 through 13.6 can be used to estimate costs for those strategies. The estimates must be based on the parameters listed above.

A query optimizer needs reasonably good estimates of these parameters, but not necessarily completely up-to-date information. In the next section, we will see how the Oracle8 database supports the collection of these parameters. The database administrator or other responsible individual must decide which statistics should be collected and when they should be updated. Although the query optimizer will obviously work better with more complete information, collecting the information is also a major expense.

Once a suitable collection of query plans has been enumerated and the costs of these plans estimated, the query optimizer can choose the best one and use it to process the query. The expense of query optimization makes it important to identify queries that are processed often. These queries can be prepared for execution by running the query optimizer once and then using the selected query plan every time the query is processed.

Finally, it is crucial to realize that the treatment of query optimization in this chapter is very simplistic. A real query optimizer offers many more strategies, must use accurate estimates of I/O speed, and must account for the CPU execution costs, which we have ignored. The Further Readings section lists sources containing careful and complete explanations of these techniques.

Query Optimization and Database Analysis in Oracle8

13.5

As the previous sections have demonstrated, effective query optimization is impossible without information about the content of the database. SQL defines the *analyze statement* to allow database administrators to trigger the collection of relevant information. Oracle8 has several tables that keep information and make it available to the optimizer.

The main point of analyzing a table is to determine the size of the table and the expected cost of scanning. For columns, we need to know the likely number of rows that satisfy range queries. For indexes, we need to be able to estimate the cost of searching. The analyze statement scans a table and its indexes and creates tables of statistics as described in Table 13.8. This information is created or updated only by explicit request from a user. As a table changes, the statistics may become outdated.

Figure 13.11 contains samples of the use of the analyze statement. Line 1 computes table and index statistics, as in Table 13.8, for the **CUSTOMER** table and all of its indexes.

Column statistics include histograms of the values. A *histogram* partitions the elements of a set into cells according to their values. Each cell contains the elements of the set whose values are in a specific range. Each cell is represented as the count of the number of elements in the cell. A bar chart is often used to display a histogram. Each bar represents a cell and the height of the bar is the number of elements in the cell.

Oracle creates histograms whose cell boundaries are chosen so that all cells have approximately the same number of elements. The approximate number of elements in each cell is the total number of rows divided by the number of cells. Unless otherwise instructed, Oracle uses 75 cells for each column. Histograms are useful only if the data in the column are not uniformly distributed. In the absence of column statistics, the optimizer assumes uniform distribution. From Table 13.8, we can see that each cell is represented by its minimum and maximum values. A histogram can be used to estimate how many rows match a range condition by multiplying the number of cells touched by the range times the number of rows per cell.

TABLE 13.8
Statistics collected by the analyze statement in Oracle8

<i>Table</i>	Number of rows
	Number of data blocks with data
	Number of unused data blocks
	Average available free space in data blocks
	Number of chained rows*
	Average row length in bytes
<i>Column</i>	Number of distinct values in the column
	Maximum and minimum values in each histogram cell
<i>Index</i>	Depth of index
	Number of leaf nodes
	Number of distinct keys
	Average number of leaf blocks per key
	Average number of data blocks per key
	Clustering factor (relative ordering of data blocks as compared with index blocks)
* A chained row is one that occupies all or part of more than one data file block. Chained rows are more expensive to fetch because more than one data block must be read.	

```
1 analyze table Customer compute statistics;  
2 analyze table Customer compute statistics  
3   for columns zipcode size 10;  
4 analyze table Customer compute statistics for table for all  
5   indexed columns;  
6 analyze table Customer estimate statistics;  
7 analyze table Customer estimate statistics sample 35 percent;  
8 analyze table Customer estimate statistics sample 20000  
9   for columns zipcode size 100, state size 10;  
10 analyze table Customer delete statistics for column firstName;
```

FIGURE 13.11
Sample analyze statements for Oracle8

Lines 2 and 3 of Fig. 13.11 compute column statistics for a single column with 10 histogram cells. Lines 4 and 5 compute table and index statistics as in line 1, as well as column statistics for indexed columns using the default number of histogram cells.

The collection of statistics can be quite expensive, especially for large tables. Often, a sample of rows can be used to make accurate estimates. For example, lines 6–9 of Fig. 13.11 use the ***estimate statistics*** option to create estimated rather than exact statistics. The first 1,064 rows of the table are sampled to create the estimates in the default case, as in line 7. Line 8 specifies that a sample of 35% of the rows should be used, and lines 9 and 10 indicate that 20,000 rows should be used in creating statistics for specific columns.

Line 10 demonstrates the use of the *analyze* statement to delete statistics. This statement deletes the column statistics for a column.

Optimizers are not perfect, and they may not be able to generate an optimal plan for each query. In reality, developers may need to modify their SQL statements to improve application performance. A developer can use the *explain plan* statement to display the query plan for an SQL query. Various formulations of the same queries can then be evaluated and the most efficient SQL incorporated in the application.

Chapter Summary

This chapter described the principles and practices of query processing. We assumed that relational tables are stored as records in files. Each query then has many different strategies, or query plans, that can be used to process it. Query optimization is the process of enumerating and evaluating query plans and choosing the best one for processing.

The cost of processing a particular query plan depends on the number of objects, the number and size of blocks of the data file and the index files, and the number of rows that match the selection and join conditions.

A range query is one with a selection of rows whose values for a particular attribute lie within some range. Range queries can be processed effectively using B+ tree indexes. Hash table indexes are useful for equality conditions and joins but not particularly helpful with range queries.

Selection based on multiple conjunctive conditions is typically processed by using an index to find all rows that satisfy conditions on one attribute and then testing those records for the other conditions. The choice of which attribute to process first is based on the likely number of rows that will satisfy the condition on that attribute. Indexing should use the attribute that has the fewest number of expected matches.

Selection based on multiple disjunctive conditions typically relies on the union of the results of separate queries. Much of the expense will come from eliminating duplicate rows.

Projection queries are easy to process, unless duplicates must be removed. One strategy that helps to eliminate duplicates is using a hashing function to partition the rows and search for duplicates within hash buckets.

Several strategies for processing join queries exist. The nested loops, block nested loops, and index nested loops join strategies are all based on selecting rows

from one table and searching the other table for matches. In a sort-merge join, both tables are sorted by the join attribute and then merged together. Hash joins apply the same hash function to each table separately and compare rows in corresponding buckets to identify matches.

Query optimization is the process that is used to find a reasonably good execution plan for a query. It begins by translating an SQL query into a relational expression. Next, equivalent expressions are found by applying the equivalence rules of the relational algebra. The assignment of execution strategies to each relational operator in an expression results in a query plan.

The optimizer must associate a cost estimate with each query plan and choose the plan with the lowest estimate. The estimates take into account a variety of statistics about the content and structure of the database tables.

Commercial databases include query optimizers that use all of the principles of this chapter to find good queries. The `analyze` statement instructs the database to collect statistics about tables, indexes, and columns for use in optimization. In addition, various query monitoring tools are available to collect and report on the performance of specific queries. The `explain plan` statement is used to determine which plan the optimizer chooses for a query.

Key Terms

Analyze statement. An SQL statement that directs a database server to collect statistics about tables, indexes, and columns.

Estimate statistics option. An option in the `analyze` statement that uses a sample of rows to estimate the information about an object.

Explain plan statement. An SQL statement that causes the query optimizer to report the query plan that it chooses for a particular select statement.

Hash join. A strategy for processing a join query in which a hash table is created from the join attributes of each table. The corresponding buckets from the two hash tables are searched for join matches.

Histogram. A partitioning of the elements of a set into cells according to their values in which each cell contains the elements of the set whose values are in a specific range. Each cell is represented as the count of the number of elements in the cell.

Nested loops join. A strategy for processing a join query in which an outer loop reads records from one file and an inner loop finds all matching records from another file. The inner loop may use indexes to improve the join performance.

Partition. The division of a collection into disjoint sets that satisfy some partitioning condition. For query processing, the typical partitioning condition is that any two records that have the same search key are placed in the same partition.

Pipelining. The process of using the output of one relational operation as input to another operation without storing that output in a file.

Query optimizer. A program that translates an SQL query into a query plan that it determines is the best plan for the query.

Query plan. A particular strategy for reading the indexes and files to process a query.

Query processing. The execution of queries in a database server.

Scan. Reading a sequence of blocks of records from a file, in linked-list or sequential order.

Sort-merge join. A strategy for processing a join in which both tables are sorted on the join attribute and then merged to create the result set.

Exercises

1. What is the difference between a relational algebra expression and a query plan? How are the two similar?
2. Describe the basic strategy for estimating the cost of execution for a query plan. What are the main cost factors in query execution? What information about the database is required to estimate the cost of query execution?
3. Give an algorithm for processing a range query for table R and attribute a for each of the following situations:
 - a. Attribute a is the primary key of R , and R is stored as an indexed sequential file, sorted and indexed on attribute a .
 - b. Attribute a is a non-key attribute of R , and R has a B+ tree index on attribute a .
 - c. Attribute a is a non-key attribute of R , and R has a hash table index on attribute a .
 - d. Attribute a is a non-key attribute of R , and R has no index on attribute a .
4. Give an algorithm for processing a projection of table R on attribute a for each of the following situations:
 - a. Attribute a is the primary key of R .
 - b. Attribute a is a non-unique attribute of R , and R has a B+ tree index on attribute a .
 - c. Attribute a is a non-unique attribute of R , and R has a hash table index on attribute a .
 - d. Attribute a is a non-unique attribute of R , and R has no index on attribute a .
5. Define and compare the performance of a block nested loops join, an indexed nested loops join, and a hash join. For each join strategy, describe a situation in which it should be used.
6. What is the purpose of converting SQL queries into relational algebra expressions before performing query optimization?
7. Define equivalence of relational expressions. Why is equivalence important to query optimization?

8. Translate each of the following SQL queries into a relational algebra expression or expression tree:
 - a. **select** lastName, ssn **from** Employee
 - b. **select distinct** lastName, firstName **from** Customer
 - c. **select * from** Employee **where** firstName = 'Jane'
 - d. **select * from** TimeCard **where** ssn = '376-77-0099' **and** date > '01-mar-1998'
 - e. **select * from** Customer **order by** accountId **desc**
 - f. **select * from** Employee, TimeCard **where** Employee.ssn = TimeCard.ssn
 - g. **select * from** Customer **order by** lastName, firstName
 - h. **select** title, genre **from** Movie, PreviousRental **where** Movie.movieId = PreviousRental.movieId **and** accountId = 101
 - i. **select** movieId, title **from** Movie m, PurchaseOrderDetail p, PurchaseOrder d, Supplier s **where** m.movieId=d.movieId **and** d.id = p.id **and** p.supplierId = s.supplierId **and** s.state='LA'
 - j. **select * from** Customer **where exists select** accountId **from** Rental
 - k. **select** lastName, firstName, title, dateRented **from** Movie, Videotape, Rental, Customer **where** Movie.movieId = Videotape.movieId **and** Customer.accountId = Rental.accountId **and** Videotape.videoId = Rental.videoId **and** dateRented > '1998-dec-1'
 - l. **select** Customer.firstName, Customer.lastName, Movie.title, PreviousRental.dateRented **from** (Movie **join** Videotape **on** Movie.movieId = Videotape.movieId) **join** (Customer **join** PreviousRental **on** Customer.accountId = PreviousRental.accountId) **on** Videotape.videoId = PreviousRental.videoId **where** (((PreviousRental.dateRented)>#12/1/98#))
9. Create a query plan for each query of Exercise 8.
10. Consider processing the query **select * from R join S on R.a=S.b**. The cost metric is the number of I/O pages. The cost of writing the final result should be uniformly ignored. Assume the following facts about the relations:
 - Relation R contains 10,000 tuples with 10 tuples per page.
 - Relation S contains 2,000 tuples with 10 tuples per page.
 - Attribute b of relation S is the primary key for S.
 - Both relations are stored in simple, unsorted files.
 - Neither relation has any indexes.

- a. What is the cost of executing the query using a simple nested loops join? What is the minimum number of buffer pages required to achieve this cost?
 - b. What is the cost of executing the query using a block nested loops join using two buffer pages (one for each relation)?
 - c. What is the cost of executing the query using a block nested loops join with unlimited buffer pages? What is the minimum number of buffer pages required to achieve this result? Justify your answers.
 - d. How many tuples will the query produce, at most, and how many pages will be required to store the result on disk? Explain.
 - e. Which of your answers to (a)–(d) will change if $R.a$ is a foreign key that refers to $S.b$?
11. Suppose that unclustered B+ tree indexes exist on $R.a$ and $S.b$, with 100 keys per page and 52 buffer pages available.
- a. How deep is the B+ tree index and how many leaf nodes exist?
 - b. What is the cost of executing the query using an indexed nested loops join? Explain.
 - c. How does the cost calculated in (b) change if only five buffer pages are available? Explain.
 - d. How would you arrange the files on disk if you had several disks available? What effect would this arrangement have on the processing?
12. Consider the following relational schema and SQL query:
- ```
Suppliers (sid: integer, sname: char(20), city: char(20))
Supply (did: integer, pid: integer)
Parts (pid: integer, pname: char(20), price: real)
Select s.sname, p.pname from Suppliers S, Parts P, Supply Y
where S.sid=Y.sid and Y.pid= P.pid and S.city='Madison' and
P.price<= 1000
```
- a. Draw a relational algebra tree that is a direct representation of the query. Improve the tree using the equivalence rules of Table 13.7.
  - b. What information about these relations will the query optimizer need to select a good query execution plan for the query?
  - c. What indexes might be of help and why?
  - d. What effect would adding **distinct** to the select clause have?
  - e. What effect would adding **order by sname** to the query have?
13. Using a database that you can access, answer the following questions:
- a. What are the system's capabilities for analyzing queries?
  - b. Write and execute an analyze statement for a table in the system. What are the results of the analysis?
  - c. What are the capabilities of the explain plan statement?

- d. Write and execute an explain plan statement for a complex query of the database. What are the results?
14. Construct a query plan for the query of Fig. 13.9. Estimate the cost of your plan using the table characteristics of Table 13.1.
15. Repeat Exercise 14 for the query of Fig. 13.10a.
16. Repeat Exercise 14 for the query of Fig. 13.10b.
17. Draw two additional query trees that are equivalent to the trees of Figs. 13.9 and 13.10.

### Further Readings

Graefe's survey [Gra93] of query-processing techniques contains an extensive bibliography. The developers of System R at IBM were among the first to use cost-based optimization, as discussed in Astrahan et al [Ast76]. The textbook by Ramakrishnan and Gehrke [RaGe99] has extensive discussions of implementation issues, including two chapters on query processing and query optimization. Cosequential processing for merging is explained in detail in Folk, Zoellick, and Riccardi [FZR98].

The features of Oracle8 that support query processing and optimization are described in Loney's *Oracle8 DBA Handbook* [Lon98]. It includes an extensive discussion of the effect of physical organization on query performance, as well as details about the statistics that are collected by analyze and explain plan statements.